

Programação em Python

Unidade 3

Desenvolver Aplicações Back-end para Web com Python

Sessão 05

Pyhton

Arquivos

- Tipos de arquivos
 - Texto
 - Sequência de caracteres armazenados de forma permanente em uma unidade de disco.
 - Binário
 - Sequência de caracteres especiais que não podem ser lidos como texto.
- O módulo “io.py” permite a manipulação de arquivos.



Saiba mais

- Sobre arquivos, visite:
<https://docs.python.org/pt-br/3.12/library/io.html#text-i-o>
- No w3schools, visite:
https://www.w3schools.com/python/python_file_handling.asp

Python

Manipulação de arquivo-texto

- Principais funções de manipulação de arquivos
 - `open()` – permite a abertura/criação de arquivos
 - `close()` – permite o fechamento de arquivos
 - `read()` – permite a leitura de um caractere contido no arquivo
 - `readLine()` – permite a leitura de uma linha do arquivo
 - `write()` – permite a escrita em uma arquivo



Saiba mais

- Sobre arquivos, visite:
<https://docs.python.org/pt-br/3.12/library/io.html#text-i-o>
- No w3schools, visite:
https://www.w3schools.com/python/python_file_handling.asp

Python

Manipulação de arquivo-texto

- Função `open()` aceita os seguintes parâmetros:
 - “r” – Abre o arquivo para leitura, gerando um erro caso ele não exista
 - “w” – Abre o arquivo para escrita, criando-o caso ele não exista
 - “a” – Abre o arquivo para a inclusão de novos caracteres, criando-o caso ele não exista
 - “x” – Cria um arquivo, gerando um erro caso ele já exista
 - “t” – Indica que o arquivo é do tipo Texto
 - “b” – indica que o arquivo é do tipo Binário
- Ex.:
`arquivo = open('meuArquivo.txt', 'wt')`
`arquivo.close()`



Saiba mais

- Sobre manipulação de arquivo-texto, visite:
<https://www.pythontutorial.net/python-basics/python-read-text-file/>
- No w3schools, visite:
https://www.w3schools.com/python/python_file_handling.asp

Python

Manipulação de arquivo-texto

- Funções de leitura:
 - `read([size])` – leitura de n bytes ou todo o arquivo
 - `readLine()` – leitura de um linha
 - `readLines()` – leitura de n linhas
- Função de escrita:
 - `write([string])` – gravação de caracteres
- Ex.:

```
f = open('meuArquivo.txt', 'w')
f.write("Olá")
f.close()
arquivo = open('meuArquivo.txt', 'r')
print(arquivo.read())
arquivo.close()
```



Saiba mais

- Sobre manipulação de arquivo-texto, visite:
<https://www.pythontutorial.net/python-basics/python-read-text-file/>
- No w3schools, visite:
https://www.w3schools.com/python/python_file_handling.asp

Pyhton

Manipulação de arquivo-texto

- Outras formas de ler um arquivo-texto:
- Ex.:

```
with open('meuArquivo.txt', 'r') as arquivo:  
    print(arquivo.read())
```

Obs.:

O arquivo será fechado após a leitura.

Pyhton

Exceções

- São imprevistos que possam ocorrer durante a execução de um programa.
- Podem ou não ser tratados pelo programa.
- Existem formas e mecanismos de tratamento para uma exceção.
- Várias linguagens permitem tratar exceções.
- Em Python, as palavras-chave `try`, `except`, `else` e `finally` são utilizadas para tratamento de exceções.



Saiba mais

- Sobre exceções, visite:
<https://docs.python.org/pt-br/3/tutorial/errors.html>

Pyhton

Tratamento de exceções

- Existem diversas classes de exceção.
- Podem ser lançadas por um módulo ou função, através da palavra-chave **raise**.
- Quando é possível tratar uma exceção:
 - Ao tentar abrir um arquivo não existe.
 - Quando uma conexão de rede deixa de funcionar.
 - Em uma tentativa de divisão por “0”.
 - Ao manipular operandos fora da faixa especificada.



Saiba mais

- Sobre exceções, visite:
<https://docs.python.org/pt-br/3/tutorial/errors.html>

Pyhton

Entendendo o tratamento de exceções

- Se uma instrução lança uma exceção, esta exceção é lançada para o chamador.
- Caso a exceção não seja tratada no chamador, ela é lançada para quem o chamou.
- Este processo continua até que a exceção seja tratada.
- Se não for tratada por ninguém da pilha de chamadas, o programa terminará anormalmente, gerando um erro de tempo de execução.



Saiba mais

- Sobre exceções, visite:
<https://docs.python.org/pt-br/3/tutorial/errors.html>

Pyhton

Tratamento de Execeções

```
dividendo = int(input('Informe um número inteiro como dividendo: '))  
divisor = int(input('Informe um número inteiro como divisor: '))
```

```
try:  
    divisao = dividendo / divisor  
except ZeroDivisionError as e:  
    print('Mensagem de erro da exceção: ' + str(e))  
    mensagem = 'O divisor não pode ser zero!'  
else:  
    mensagem = "%d divisor por %d igual a %.2f" % (dividendo, divisor, divisao)  
finally:  
    print('Finalizei a estrutura try/catch/else/finally com sucesso!')
```



```
print(mensagem)
```

Pyhton

Manipulação de pastas e arquivos

- O módulo “`os.py`” permite o acesso ao ambiente do sistema operacional.
- Alguns métodos:
 - `os.getcwd()` – recuperar o diretório de trabalho atual.
 - `os.path.abspath()` – recuperar o diretório absoluto de um arquivo.
 - `os.path.exists()` – verificar se um arquivo ou pasta existe.
 - `os.path.isdir()` – verificar se o nome informado é uma pasta.
 - `os.path.isfile()` – verificar se o nome informado é uma pasta.
 - `os.listdir()` – retornar uma lista com os arquivos e pastas de uma pasta informada.



Saiba mais

- Sobre manipulação de pastas e arquivos, visite: <https://docs.python.org/pt-br/3.12/library/os.html>

Python

- Considerações finais.
- Agradecimentos.



Programação em Python

Botafogo



Até a próxima!

